Eötvös Loránd Tudományegyetem

Informatikai Kar

Algoritmusok és Alkalmazásaik Tanszék

Cache-optimális algoritmusok elemzése

Szabó László Ferenc
Docens

Nagy Péter
Programtervező Informatikus MSc

Budapest, 2023

Tartalomjegyzék

1. Bevezetés **3**

2. Cache modellje **4**

2.1. Csere stratégiák 5

2.1.1. Online algoritmusok 6

2.1.2. Verem stratégiák 6

2.1.3. OPT 7

2.1.4. LRU 8

2.1.5. FIFO 8

2.2. Asszociativitás 9

2.3. Írási stratégia 10

3. Gyorsítótár független algoritmusok **12**

3.1. Lista 12

3.1.1. Másolás 12

3.1.2. Partíció 13

3.1.3. Leszámláló rendezés 14

3.2. Mátrix szorzás 15

3.3. Rendezés 15

3.4. FFT 16

3.5. Diff.egyenlet közelítés 16

3.6. Statikus keresőfa 16

3.7. Edénysor szita 16

3.7.1. A rekurzív fa 16

3.7.2. Feladat 17

3.7.3. A megoldás 17

3.7.4. Műveletigény gyorsítótár nélkül 19

3.7.5. Cache-oblivious edényszita 22

3.7.6. Műveletigénye gyorsítótárral 22

3.7.7. Feltételek a gyorsítótár méretére 22

3.7.8. Tetszőleges n-ig, nem csak teljes fára 22
3.7.9. Az igazi program fix méretű számokkal 22
3.7.10. Az igazi program változó méretű számokkal 22

4. Szimulátor **23**
4.1. Igazi cache 23
4.2. Megoldás 23
4.3. Választható cache-ek 23

5. Mérési eredmények **24**

6. Irodalomjegyzék **25**

1. fejezet

Bevezetés

A modern számítógépek olyan sebességeket érnek el, hogy a jelterjedés fizikai korlátai már egy integrált áramkörön belül sem feltétlenül engedik meg az összes áramköri elem szinkron működést. A végrehajtó egység és a memória sebessége között már nagyságrendi különbségek vannak.

A probléma kezelésére gyorsítótárakat vezettek be a végrehajtó egység és a memória közé, ami az utoljára használt memóriaelemeket tárolja. A gyorsítótár sebessége nagyobb mint a memóriáé, de lassabb, mint a végrehajtó egységé. Egy modern számítógépben a gyorsítótáraknak hierarchiája is van, ahol egy gyorsítótár egy szintben alatta lévő másik gyorsítótár elemeit tárolja.

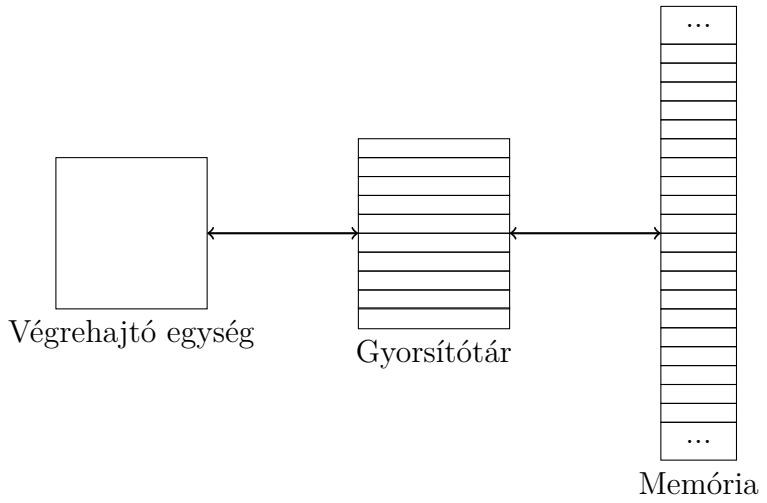
A gyorsítótárakkal ellátott számítógép legjobb kihasználásához olyan algoritmusokat kell tervezni, amik figyelembe veszik a gyorsítótárak hierarchiáját.

2. fejezet

Cache modellje

Az ideális gyorsítótár modell[1] egy végrehajtó egységből, memóriából, és egy gyorsítótárból áll. A végrehajtó egység a memóriát csak a gyorsítótáron keresztül éri el.

A végrehajtó egység a memóriát gépi szavanként éri el, ez egy kicsi konstans, például 8 bájt. Egy (Z, L) gyorsítótár L hosszú gyorsítótársorokból áll, és Z/L sort tartalmaz. Egy sor L gépi szót tárol, ezek a memóriában folytonosan helyezkednek el. Egy sor címe a memóriába csak a sorhossz többszöröse lehet. A gyorsítótár a memóriát gyorsítótársoronként tárolja. Egy sor a gyorsítótárban a memória bármelyik szakaszát tárolhatja.



2.1. ábra. Az ideális gyorsítótár modell

Amikor a végrehajtó egység el akar érni egy adott szót a memóriában, a gyorsítótárhoz fordul. Ha a tárban van olyan sor, ami tartalmazza a kért gépi szót, akkor a gyorsítótár a kérést azonnal kiszolgálja. Ezt gyorsítótár-találatnak nevezzük. Ha a tár egyik sora se tartalmazza a kért szót, akkor a szóhoz tartozó sort a tár be kell olvassa a memóriából, majd ezután tudja teljesíteni a kérést. Ezt gyorsítótár-hibának

nevezzük.

Ha a tárban nincs üres sor amikor egy új sort be kell olvasni, akkor először egy sort ki kell ürítsen. Ha az ürítésre kiválasztott sor módosult a hozzá tartozó memóriához képest, akkor azt először vissza kell írnia a memóriába. Miután a sort szinkronizálta a memóriával, az új sorral felülírja az ürített sort.

A gyorsítótár-csere stratégiája diktálja, hogy a tár hogyan választja ki a sort, amit ürít. Az ideális gyorsítótár modell ürítési stratégiája az optimális offline stratégia[2]. Az optimális stratégia azt a sort üríti, amit a jövőben legkésőbb ér el a program a teljes futása során. Ez a stratégia használja ki a legjobban az időbeli lokalitást.

A gyorsítótár modellben az időbonyolultság az algoritmus futása alatt végzett elemi műveletek száma, a memóriaelérésekkel együtt. A gyorsítótár-bonyolultság a futás alatti találati hibák száma.

Egy algoritmus gyorsítótár-tudatos, ha van olyan fordítási vagy futási idejű paramétere, ami a gyorsítótár méretétől függ. Ha nincs ilyen paramétere, akkor az algoritmus gyorsítótár-független.

2.1. Csere stratégiák

Több csere stratégia is ismert:

- MIN, minimális, OPT, optimális, vagy Bélády algoritmusa: Ezt az elemet cseréli, amelyiket a jövőben legkésőbb éri el az algoritmus. Ennek eldöntésére szükség van a teljes memóriaelérési sorozatra.

- LRU, least recently used, legrégebben használt: Azt a sort cseréli, amit a legrégebben ért el az algoritmus. Ennek eldöntésére elég a csere pillanatáig ismerni az elérési sorozatot. Az LRU heurisztikája az a feltételezés, hogy az időben közelebbi elért memóriát valószínűleg újra el kell érni.

- LFU, least frequently used, legritkábban használt: A gyorsítótár nyilván tartja, hogy egy sort hányszor ért el az algoritmus mióta a gyorsítótárba került. Azt az elemet cseréli, amelyik elérési száma a legkisebb.

- FIFO, first in, first out, első be, első ki: Azt a sort cseréli, amelyik legrégebben került a tárba. A stratégia előnye az egyszerű megvalósíthatóság.

- Random, véletlen: A cserét véletlenszerűen választja ki. Az előnye az egyszerű implementáció.

Ha S egy gyorsítótár-csere stratégia, akkor legyen S(k) egy olyan gyorsítótár, ami az S stratégiát használja, és k sort tartalmaz.

2.1.1. Online algoritmusok

A gyakorlatban megvalósítható gyorsítótár-csere stratégiák online algoritmusok. Az online algoritmusok a bemenetüket sorban, egyesével kell feldolgozzák, a teljes bementet nem látják egyszerre. Egy gyorsítótár-csere algoritmusnak minden memória-eléréskor úgy kell döntést hozzon a cseréről, hogy csak a múltbeli elérések sorozatát ismeri.

Egy offline algoritmus a teljes bemenethez hozzáfér a teljes futása során.

A stratégiák elemzését egy egyszerűbb modellben[5] végezzük. Ezek a szabályok az igény szerinti lapozást írják le. Legyen $A = [1, 2, ..., n]$ az elérhető elemek halmaza. Legyen X egy A feletti véges sorozat, az elérések sorozata. Legyen $B_t(k)$ az $S(k)$ gyorsítótár tartalma X t hosszú prefixének feldolgozása után. Minden k-ra:

- Legyen $B_0(k) = \emptyset$. Tehát a gyorsítótár üres az algoritmus futása előtt.
- Ha $t > 0$, és $X_t \in B_{t-1}(k)$, akkor legyen $B_t k = B_{t-1}(k)$. Azaz találat esetén a gyorsítótár nem változtatja meg, hogy mely elemeket tartalmazza.
- Ha $t > 0$, $X_t \notin B_{t-1}(k)$, és $|B_{t-1}(k)| < k$, akkor legyen $B_t k = B_{t-1}(k) \cup \{X_t\}$. Ha van szabad hely a gyorsítótárban, akkor találati hiba esetén csere nélkül hozza be a memóriaelemet a tárba.
- Ha $t > 0$, $X_t \notin B_{t-1}(k)$, és $|B_{t-1}(k)| = k$, akkor legyen $y_t \in B_{t-1}(k)$, és $B_t(k) = B_{t-1}(k) \setminus y_t \cup \{X_t\}$. Találati hiba esetén ha nincs üres sor a tárban, akkor egy elemet kicserél, hogy behozhassa az elérni kívántat. A cserélendő elem az y_t, ezt a csere stratégia választja ki.

Ebben a modellben a gyorsítótárban nem sorok vannak, hanem A elemei. Ezek a gyorsítótár modellben a memória azon tartományai, amik gyorsítótár sorok lehetnek. Valamint az elérésben nem különböztetjük meg az olvasást és az írást.

2.1.2. Verem stratégiák

Egy S gyorsítótár-csere stratégiát verem algoritmusnak[5] nevezünk, ha teljesíti a következő feltételt. Legyen X egy memória-elérési sorozat. S verem algoritmus, ha minden $t \in 1, 2, ..., |X|$, és minden $k \in \mathbb{N}$-re

$$B_t(k) \subset B_t(k+1)$$

Azaz ha minden időpontban egy nagyobb gyorsítótár tartalmazza az összes kisebb gyorsítótár elemeit.

A verem algoritmusnak egy ekvivalens definíciója, ha S X futtatása közben minden $t \in 1, 2, ..., |X|$ időpontra ad egy egyértelmű preferencialistát az $\{X_1, X_2, ..., X_t\}$-ben előforduló elemekre, a gyorsítótár méretétől függetlenül, és egy $S(k)$ gyorsítótár csere esetén $B_t(k)$ elemei közül a legkisebb precedenciájút cseréli. A precedencialista nem feltétlen kell megegyezzen minden időpontban.

Egy verem algoritmust használva a gyorsítótár méretét növelve a találtai hibák száma nem nő. Nézzünk t időpontban egy gyorsítótár találatot $S(k)$-ban. Mivel $B_t(k) \subset B_t(k+1)$, ezért az elérés $S(k+1)$-ben is találat. Tehát minden k-nál nagyobb gyorsítótárban is találat az elérés.

Az LRU verem algoritmus.

Az OPT verem algoritmus.

Az LFU verem algoritmus, feltéve, hogy az egyenlő frekvenciájú elemekre minden időpontban létezik egy egyértelmű precedencia.

A FIFO nem verem algoritmus.

A véletlen startégi nem feltétlen verem algoritmus. Ha egyenletesen véletlenül választjuk ki egy $S(k)$ gyorsítótár elemét cserére, akkor pozitív a valószínűsége annak, hogy az $S(k+1)$ tár egy másik elemet választ. Ha a t időpontban a tár méretétől függetlenül egyenletes véletlenül kiválasztjuk $X_1, X_2, ..., X_t$ egyedi elemeinek egy permutációját, és ezt precedencialistaként használjuk a cserélt elem kiválasztására, akkor a véletlen stratégia verem algoritmus. Ez az RR stratégia. Egy adott $RR(k)$ gyorsítótárban egy elem ürítésének esélye $1/k$, tehát egyenletes eloszlású.

2.1.3. OPT

Az optimális stratégiánál jobb eredményt lehet elérni, ha nem követeljük meg, hogy egy elérést csak a gyorsítótárból lehet kiszolgálni. Nézzük a $X = (1, 2, 3, 1, 2)$ elérési sorozatot, a $t = 3$ időpontot, és egy kételemű gyorsítótárat. A gyorsítótár ekkor tele van, és $B_2(2) = \{1, 2\}$.

Nézzük az $OPT(2)$ stratégiát. A 3 tárba hozatalához egy elemet ki kell cserélnie. Cserére a 2-t választja, mert ez az elem lesz legkésőbb újra elérve. Így $B_3(2) = \{1, 3\}$. $t = 4$-ben az elérés találat, de $t = 5$-ben 2 már nincs a tárban, és találati hibát okoz. Tehát X kiszolgálásához 4 memóriaműveletre volt szükség, a $t \in \{1, 2, 3, 5\}$ időpontokban.

Nézzünk egy olyan optimális stratégiát, ahol csere nélkül, közvetlenül a memóriából is ki lehet szolgálni egy elérést. Ez a stratégia $t = 3$-ban a 3 elérést a memóriából közvetlenül szolgálja ki, és $B_3(2) = B_2(2) = \{1, 2\}$. Ekkor X kiszolgálásához elég 3 memóriaművelet, a $t \in \{1, 2, 3\}$ időpontokban.

2.1.4. LRU

Egy adott problémára a versenyképességi analízisben egy A online algoritmust hasonlítunk össze az O optimális offline algoritmussal. Legyen X egy inputsorozat, $A(X)$ az A műveletigénye X-en, és $O(X)$ az O műveletigénye X-en. Ekkor A k-versenyképes[6], ha létezik egy C konstans, hogy minden X inputsorozatra

$$A(X) \leq kO(X) + C$$

Ha $C = 0$, akkor A szigorúan k-versenyképes. A legkisebb ilyen k az A versenyképességi aránya.

Slater és Tarjan[7] megmutatta, hogy az LRU(k) az OPT(k^*)-hoz képest $k/(k - k^* + 1)$ versenyképes. Ha $k = k^*$, akkor az LRU k-versenyképes az OPT-hoz képest. Ha $k = 2k^*$, akkor az LRU(k) az OPT(k*)-hoz képest

$$k/(k - k^* + 1) = 2k^*/(2k^* - k^* + 1) = 2k^*/(k^* + 1) < 2$$

versenyképes[1].

Legyen A egy adott algoritmus. Legyen $LRU(n, Z, L)$ a gyorsítótár komplexitása A-nak egy n méretű bemeneten és egy LRU(Z, L) gyorsítótáron. Legyen $OPT(n, Z, L)$ a gyorsítótár komplexitása A-nak egy n méretű bemeneten és egy OPT(Z, L) gyorsítótáron. Ekkor az $LRU(n, Z, L)$ gyorsítótár komplexitás reguláris[1], ha

$$LRU(n, Z, L) = O(LRU(n, 2Z, L))$$

Tegyük fel, hogy A egy reguláris gyorsítótár komplexitású algoritmus. Ekkor

$$OPT(n, Z, L) \leq LRU(n, Z, L) = O(LRU(n, 2Z, L)) = O(OPT(n, Z, L)$$

az optimalitást, a regularitást, és a 2-versenyképességet felhasználva. Tehát

$$LRU(n, L, Z) = \Theta(OPT(n, L, Z)$$

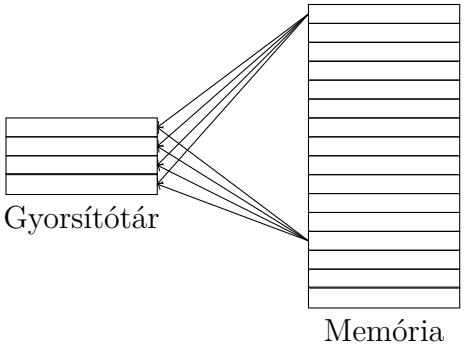
2.1.5. FIFO

Bélády[3] megmutatta, hogy FIFO stratégiára létezik olyan k és K, $k < K$, és egy elérési sorozat, hogy ezen az elérési sorozaton a FIFO(K) több találati hibát eredményez, mint a FIFO(k). Fornai és Iványi megmutatta[4], hogy a hiba arányának nincs felső korlátja.

2.2. Asszociativitás

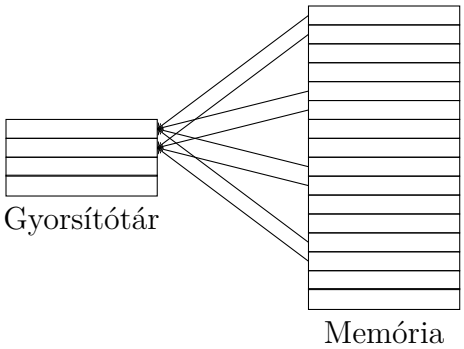
Az ideális gyorsítótár modellben a memória egy sorát a gyorsítótár bármelyik sorába be lehet hozni. Ezt hívják teljesen asszociatív gyorsítótárnak[8].

A gyakorlatban egy teljesen asszociatív gyorsítótár csak kevés sorra valósítható meg hatékonyan, mert minden elérésnél a gyorsítótárnak meg kell keresnie a sorok között, hogy tárolja-e az elérni kívánt sort.



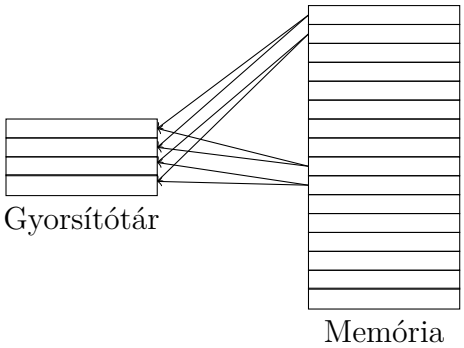
2.2. ábra. Teljesen asszociatív gyorsítótár

Közvetlen leképezésű gyorsítótárban[8] a memória egy sora a tárban csak egyetlen sorba hozható be. Ez a csere stratégiát is meghatározza, a beolvasni kívánt sor címe meghatározza a cserélt sort. Általában egy sor címét a gyorsítótárban a sor címének alsó bitjei határozzák meg, így a gyorsítótárba lehet hozni a memória egy egybefüggő szakaszát.



2.3. ábra. Közvetlen leképezésű gyorsítótár

Csoport-asszociatív gyorsítótárban[8] a memória egy sora a gyorsítótár több sorába is behozható, de nem az összesbe. A gyorsítótár több, egymástól független teljesen asszociatív gyorsítótárból áll. A memóriasor címe meghatározza, hogy a sor melyik asszociatív tárba kerül, és az asszociatív táron belül a csere stratégia alapján kerül elhelyezésre. Ha egy memóriasor A különböző gyorsítótár sorba kerülhet, akkor azt A-utas gyorsítótárnak nevezzük.



2.4. ábra. Csoport-asszociatív gyorsítótár

Tegyük fel, hogy a gyorsítótár paraméterei kettő-hatványok. A cache $Z = 2^z$ bájt, egy sor $L = 2^l$ bájt, és a cache $A = 2^a$ asszociatív, azaz egy memóriasor A gyorsítótár sorba kerülhet. Ekkor egy memóriabájt címének bitjeit egy lehetséges felosztása:

- Az l darab legkisebb helyi-értékű bit a bájt címe az őt befoglaló sorban.
- A következő $z-a$ helyi-értékű bit határozza meg, hogy a memóriasor a $C/(AL)$ darab teljesen asszociatív gyorsítótár közül melyikhez tartozik.
- A cím maradék bitjei meghatározzák, hogy egy teljesen asszociatív gyorsítótárhoz hány különböző sor tartozik.
- Ha $A = C/L$, akkor a gyorsítótár teljesen asszociatív.
- Ha $A = 1$, akkor a gyorsítótár közvetlen leképezésű.
- Ha $1 < A < C/L$, akkor a gyorsítótár A-utas.

2.3. Írási stratégia

A gyorsítótáraknak az írási műveleteket is kezelniük kell. Erre több különböző megoldás ismert[8].

Egy ha az írás olyan sort érint, amit a gyorsítótár tartalmaz, akkor az írási találat. Ilyenkor két megoldás közül lehet választani.

- A visszaírási (write back) stratégia az írást csak a gyorsítótárban végzi el, és az érintett sort megjelöli módosítottnak. Egy módosított sort a csere előtt a memóriába ki kell írni. Ezzel a megoldással memóriaműveletet lehet spórolni, ha ugyanarra a memóriasorra rövid időn belül több írás is történik. Ez a lehetséges hatékonyságnövelés növeli a megvalósítás áramköri komplexitását.

- A átmenő (write through) stratégia az írást a gyorsítótárban és a memóriában is elvégzi. A stratégia megvalósítása egyszerűbb, de minden írási művelet memóriaművelettel jár.

Ha az írási találati hibát is kétféleképpen kezelheti a gyorsítótár.

- Foglalással (write allocate) a gyorsítótár sorcserével a tárban is elhelyezi az írt sort. Ha az írás nem fedi le az egész memóriasort, akkor a gyorsítótár először az egész sort ki kell olvassa a memóriából, az olvasási találati hibához hasonló módon.

- Foglalás nélkül (no write allocate) a gyorsítótár az írást közvetlenül a memóriában végzi el, a saját tárát megkerülve. Ha a gyorsítótár ki kell tudjon szolgáljon olyan írásokat, amik nem a teljes memóriasort módosítják, akkor a memóriának támogatnia kell a memóriasornál finomabb elérési műveleteket.

Ezeknek a lehetőségeknek minden kombinációja megvalósítható, különböző bonyolultsággal.

- A módosított gyorsítótár sorok megnövelik a gyorsítótár állapotainak számát, és a műveletek állapotgépeinek állapotait is.

- A közvetlen memóriaírás a gyorsítótár megkerülésével szükségessé teszi, hogy a memóriát ténylegesen olyan finomsággal lehessen címezni, amilyen finoman a végrehajtó egység logikailag tudja címezni.

- Ha vannak párhuzamos gyorsítótárak, akkor ezek a megoldások a gyorsítótár koherenciára adott megoldást is befolyásolják. Párhuzamos gyorsítótárak előfordulhatnak egy végrehajtó egységgel is, például külön adat és utasítás gyorsítótár, és több végrehajtó egységgel rendelkező rendszerben is.

3. fejezet

Gyorsítótár független algoritmusok

3.1. Lista

Legyen A egy n elemű, N bájt hosszú tömb. A tömb elemei a memóriában folytonosan legyenek elrendezve. A feladat a tömb minden elemét sorban egyszer elérni. Ez az elérés lehet olvasás vagy írás is.

Tegyük fel, hogy a gyorsítótár mérete (Z, L), és létezik legalább egy gyorsítótársor, $Z \geq L$.

Ekkor a tömb minden elemeinek elérésének gyorsítótár bonyolultsága $O(1 + N/L)$. Az tömb eléréséhez legfeljebb 2 olyan memóriasort kell elérni, ahol a tömb nem fedi le az egész sort, a tömb elején és a tömb végén. A tömb többi eleme legfeljebb N/L memóriasort fed le teljesen.

3.1.1. Másolás

Legyen A egy n elemű, N bájt hosszú tömb. A tömb elemei a memóriában folytonosan legyenek elrendezve. A feladat a tömb minden elemét egy új memóriacímre másolni. A másolás történhet a tömbben előre és hátrafele is, attól függően, hogy a tömb régi és új helye hogyan fed át.

Tegyük fel, hogy a (Z, L) méretű gyorsítótárban legalább 2 gyorsítótársor van.

Ekkor az egyik gyorsítótársort az írásra használva, és a másik sort az olvasásra használva a másolás elvégezhető $2O(1 + N/Z) = O(1 + N/Z)$ memóriaművelettel.

3.1.2. Partíció

Legyen A egy n elemű, N bájt hosszú tömb. A tömb elemei a memóriában folytonosan legyenek elrendezve. Legyen T eldönthető tulajdonság, ami értelmezett a tömb minden elemére.

Rendezzük át helyben a tömböt úgy, hogy az összes olyan elem, amire a T tulajdonság hamis, a tömb elejére kerül, és azok az elemek, amikre a T tulajdonság igaz, a tömb végére kerülnek. A felosztás eredménye legyen a hamis tulajdonságú elemek száma.

A feladat a gyorsrendezés egy részfeladata, ahol a rendezendő tömb elemeit egy tengelyelem szerint két részre osztja, a tengelynél nem-nagyobbakra, és a tengelynél nagyobbakra. [9]

Algoritmus 3.1 Az $A[0 : n-1]$ tömb felosztása a T tulajdonság szerint

Require: A: a felosztandó tömb
Require: n: a tömb hossza
Require: T: egy eldönthető tulajdonság
Ensure: m: a hamis tulajdonságú elemek száma
Ensure: $A[0 : m-1]$ minden eleme hamis tulajdonságú T szerint
Ensure: $A[m : n-1]$ minden eleme igaz tulajdonságú T szerint
Ensure: az A kimenet az A bemenet permutációja

```
function FELOSZTÁS(A, n, T) → m
    l ← 0
    r ← 0
    while true do
        while (l + r < n) ∧ (¬T(A[l])) do
            l ← l + 1
        end while
        while (l + r < n) ∧ T(A[l]) do
            r ← r + 1
        end while
        if l + r ≥ n then
            return l
        end if
        t ← A[l]
        A[l] ← A[r]
        A[r] ← t
        l ← l + 1
        r ← r + 1
    end while
end function
```

Tegyük fel, hogy a gyorsítótár mérete (Z, L), és Z/L legalább akkor, hogy 3 tömbelemet a gyorsítótárban lehessen tartani, a memória bárhol elhelyezkedve. Mindig tároljuk a gyorsítótárban

- az l által vizsgált elemet,
- az r által vizsgált elemet,
- a cserénél használt ideiglenes t elemet.

Ekkor a felosztást legfeljebb $O(1 + N/L)$ memóriaművelettel el lehet végezni.

- Növekvő indexekkel legfeljebb a tömb összes memóriasorait egyszer járjuk be. Ez $O(1 + N/L)$ memóriaművelet.
- Csökkenő indexekkel legfeljebb a tömb összes memóriasorait egyszer járjuk be. Ez $O(1 + N/L)$ memóriaművelet.
- Az ideiglenes változóhoz tartozó memóriasorokat legfeljebb egyszer kell a gyorsítótárba hozni. Ez $O(1)$ memóriaművelet.

3.1.3. Leszámláló rendezés

A leszámláló rendezés egy lineáris műveletigényű, nem összehasonlításon alapuló rendezés.[9]

Legyen A egy n elemű, N bájt hosszú tömb. Legyen K egy kiszámítható függvény, ami a tömb minden elemét a $\{0, 1, ..., k - 1\}$ halmazba képzi. k legalább 2. Tipikusan k lényegesen kisebb, mint n. K az elemek kulcsát adja meg, ami szerint a tömböt rendezni szeretnénk.

A feladat a tömböt úgy átrendezni helyben, hogy sorban először az összes 0 kulcsú elem legyen, majd ezeket sorban kövesse az összes 1 kulcsú elem, és így tovább, végül a tömb végén a $k - 1$ kulcsú elemek helyezkedjenek el. A kimenetben adjuk meg minden lehetséges kulcsra azt is, hogy a tömbben hol helyezkednek el az adott kulcsú elemek.

A megoldás a tömb kétszeri bejárásával megadható.

- Inicializáljunk a $C[0 : k-1]$ tömböt nullákkal, majd a tömb egyszeri bejárásával számoljuk meg az előforduló kulcsok frekvenciáját. Ez a C tömb a kimenet része is.
- Az $S[0 : k - 1]$ tömbben számítsuk ki a kulcsokhoz tartozó kezdőelemek indexeit. $S[0] = 0$, és $\forall i \in \{1, 2, ...k - 1\} : S[i] = S[i - 1] + C[i - 1]$. Ez az C és S egyszeri bejárásával kiszámítható. S a kimenet része.
- Másoljuk le S-t T-be. A következő dupla-ciklusban a T invariánsa a következő. Minden $k' \in \{0, 1, ..., k - 1\} : \forall i \in [S[k'], T[k']) \rightarrow K[A[i]] = k'$, azaz $S[k']$ és $T[k']$ között minden elem a helyén van.

- Járjuk be sorra k'-vel a $\{0, 1, ..., k-1\}$ kulcsokat. Minden k' kulcsra biztosítsuk, hogy a k'-höz tartozó szakasz minden eleme a megfelelő helyen van. Amíg $T[k'] - S[k']$ nem egyenlő $C[k']$-vel, addig vegyük az $i = T[k']$ elemet. Ha $K[A[i]] = k'$, akkor az i-dik elem a helyén van, növeljük meg $T[k']$-t. Különben cseréljük meg $A[i]$-t és $A[K(A[i])]$-t.

A megoldásban az utolsó ciklus minden lépésben egy elemet a helyére tesz, a ciklus invariánsa szerint, és egy elemet legfeljebb kétszer vizsgál meg.

Tegyük fel, hogy a gyorsítótár mérete (Z, L). Tegyük fel, hogy k számláló K' bájttal leírható. Valamint tegyük fel, hogy Z/L legalább akkora, hogy $2k$ számlálót, és k tömbelemet, és egy ideiglenes elemet a csérhez mindig a gyorsítótárban lehet tartani, függetlenül a elhelyezésüktől a memóriában. Ekkor:

- C kiszámításához k számlálót kell a gyorsítótárban tartani, miközben a A-t sorban bejárjuk. Ez legfeljebb $O(k + K'/L + N/L)$ memória műveletet igényel.
- S kiszámításához k számlálót kell összesíteni. Ez legfeljebb $O(1 + K'/L)$ memória műveletet igényel.
- T előállítása S másolása, legfeljebb $O(1 + K'/L)$ memória művelet.
- Minden kulcs bejárása nem rosszabb, mint a teljes tömb bejárása, hiszen kulcsok szerint is sorba járjuk be, legfeljebb kihagyásokkal. A résztömbök végeit legfeljebb $O(k)$ memóriaművelettel a gyorsítótárba lehet hozni, és a köztes elemek bejárása legfeljebb $O(N/L)$ memóriaműveletet igényel. Feltettük, hogy a kulcsokhoz tartozó számlálók végig a gyorsítótárban vannak.

Tehát összesen a memóriaműveletek száma legfeljebb $O(1 + k + K'/L + N/L)$.

3.2. Mátrix szorzás

Katasztrofális. Tall cache.

Oszd meg és uralkodj.

3.3. Rendezés

Katasztrofális.

Oszd meg és uralkodj. 2 algoritmus is van. Az alsó korlát elérhető, de a papír nem az igazi erről. Tall cache assumption. A tall cache assumption lényegesen befolyásolja a műveletigényt.

3.4. FFT

Katasztrofális.

Oszd meg és uralkodj. Alsó korlát is van, amit elér, de túl erős feltevésekkel.

3.5. Diff.egyenlet közelítés

Katasztrofális.

Oszd meg és uralkodj. Több dimenzióban.

3.6. Statikus keresőfa

Katasztrofális.

Van Emde Boas fák. Dinamikus eset szuperbonyolult, asszem.

3.7. Edénysor szita

Katasztrofális.

Az edénysor szität a szakdolgozatomban írtam le. Ez egy eratoszthenészi rekurzívan szegmentált szita.

Az eratoszthenészi sziták prímek többszöröseit jelölik meg egy adott intervallumon, és egy-egy prím többszöröseinek megjelöléseit a többi prímtől függetlenül végzik. A szität lehet használni prímek keresésére is, és az intervallum számainak prím-faktorizációjára is.

Egy szegmentált szita a teljes intervallumot kisebb intervallumokra osztja, ezek a szegmensek, és a szegmensekben külön-külön végzi el a megjelöléseket. A szegmensek átfedés nélkül lefedik a teljes intervallumot. A szegmens méretet úgy választják meg, hogy szitáláshoz használt prímek és egy szegmens a gyorsítótárban elférjen. Egy szegmentált szita cache-aware.

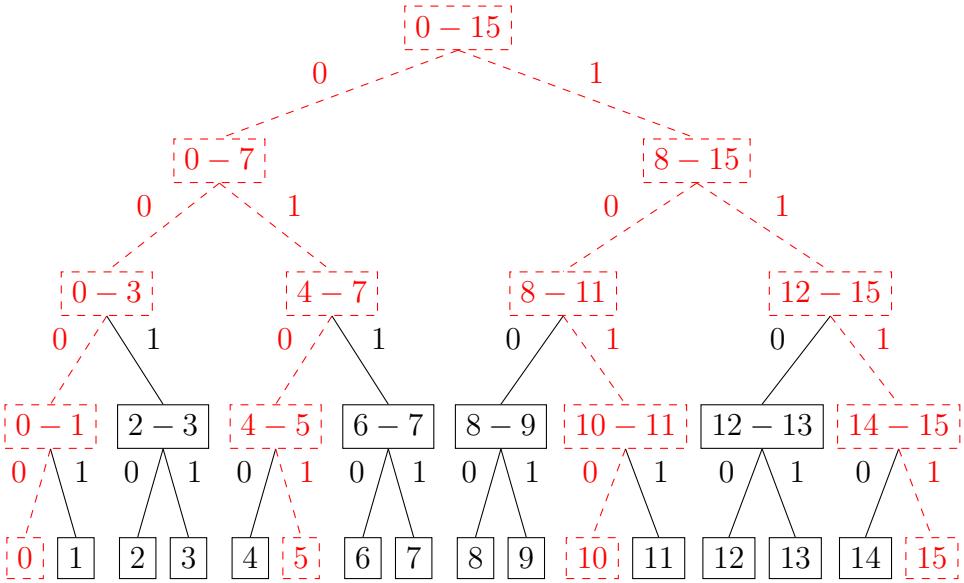
A rekurzív edényszita egy a^h hosszú intervallumot szitál, ahol a és h természetes számok, $a \geq 2$. Ha $h > 0$, akkor az intervallumot a darab ugyanolyan hosszú részre osztja, és ezeket rekurzívan szitálja. A prímek listáját előszűri, hogy egy szegmensbe csak olyan prím kerüljön, ami ténylegesen szitál is a szegmensben.

3.7.1. A rekurzív fa

Vegyük a $[u, u+a^h)$ intervallumot, u természetes szám. és egy p prímet, aminek van nullánál nagyobb többszöröse az intervallumban. Tegyük fel, hogy $h > 0$. Ekkor az i. részintervallum $[u + ia^{h-1}, u + (i+1)a^{h-1}), (i \in 0, ..., a-1)$

- Ha $p \geq a^h$, akkor p pontosan egy részintervallumban szitál.
- Ha $a^{h-1} < p < a^h$, akkor p legalább egy részintervallumban szitál, de nem feltétlen mindegyikben.
- Ha $p \leq a^{h-1}$, akkor p minden részintervallumban szitál.

Példának nézzük meg, hogy az 5 milyen utat jár be a rekurzív fában a $[0, 2^4)$ intervallum szitálásakor. Az ábrán a prím 0-t is bejárja, ami prímek keresésekor nem következik be.



3.1. ábra. Az 5 többszörösei a $[0, 2^4)$ intervallumhoz tartozó rekurzív fában.

3.7.2. Feladat

Jelöljük meg az $[u, u+a^h)$ intervallumban az összes olyan számot, amik többszörösei valamelyik $p \leq m$ prímnek.

3.7.3. A megoldás

Azt, hogy egy adott p prím melyik részintervallumban fog szitálni legközelebb, meghatározhatjuk maradékos osztással. Legyen u a szóba jöhető legkisebb intervallum kezdőpontja. Legyen $m = u \bmod p$. Ekkor p következő többszöröse

$$\begin{cases} u & \text{ha m} = 0 \\ u - m + p & \text{különben} \end{cases} \tag{3.1}$$

Az is megoldás, ha minden prím mellé tároljuk a legutolsó megtalált többszörösét is, és ezt frissítjük, amikor a prímet megjelölésre használjuk. Ezzel az osztás időigényét memóriaigényre cseréljük.

Algoritmus 3.2 Az $[u, u + a^h)$ intervallum szitálása

Require: a: a számrendszer alapja
Require: h: a fa magassága
Require: m: a keresett prímek felső korlátja
Require: u: a az intervallum kezdőpontja
Require: P: lista, azok a p prímszámok, hogy létezik p-nek többszöröse az $[u, u + a^h)$ intervallumban
Ensure: P: P kiegészítve az intervallumban talált prímekkel

```
function RekurzívSzita(a, h, m, u, P) → P
    if h = 0 then
        if P üres és u ≥ 2 then                    ▷ u egy újonnan megtalált prím
            if u ≤ m then
                P ← P ∪ {u}
            end if
        else
            jelöljük meg u-t                       ▷ u összetett szám
        end if
    else
        Legyenek a részintervallumok I_i = [u + ia^{h−1}, u + (i + 1)a^{h−1}, i ∈ [0, a)
        Q legyen listák egy a elemű tömbje, és legyen minden lista üres
        for p ∈ P do
            Legyen i a legkisebb olyan szám, hogy p-nek van többszöröse I_i-ben
            Q[i] ← Q[i] ∪ {p}
            P ← P \ {p}
        end for
        for i = 0, ..., a − 1 do
            Q[i] ← RekurzívSzita(a, h − 1, m, u + ia^{h−1}, Q[i])
            for p ∈ Q[i] do
                if p-nek van többszöröse I_{i+1}, ..., I_{a−1}-ben then
                    Legyen j a legkisebb olyan szám,
                    hogy p-nek van többszöröse I_{i+1}, ..., I_{a−1}-ben
                    Q[j] ← Q[j] ∪ {p}
                else
                    P ← P ∪ {p}
                end if
                Q[i] ← Q[i] \ {p}
            end for
        end for
    end if
    return P
end function
```

Prímek keresését a $[0, a^h)$ intervallumban a RekurzívSzita$(a, h, \lfloor\sqrt{(a^h)}\rfloor, a^h, \emptyset)$ hívás után az intervallum végigolvasásával lehet elvégezni. Azok a számok a prímek, amik nincsenek megjelölve.

A $[0, a^h)$ intervallum prím-faktorizációját a RekurzívSzita$(a, h, a^h, a^h, \emptyset)$ hívással lehet elvégezni. A hívás után az intervallum egy száma prím, ha nincs megjelölve, és összetett, ha meg van jelölve, és az összes prímosztója megjelölte.

3.7.4. Műveletigény gyorsítótár nélkül

Tegyük fel, hogy a $[0, a^h)$ intervallumot szitáljuk. Legyen $n = a^h$, és m a minket érdeklő prímek felső korlátja. Legyen $p \leq m$ egy prím.

Nézzük a rekurzív fában az összes a^k hosszú részintervallumot, $k \leq h$.

- Ha $p > a^k$, akkor p-nek legfeljebb egy többszöröse van egy adott részintervallumban, és összesen $\lfloor a^h \rfloor$ többszöröse van, és ennyi a^k hosszú részintervallumot jár be.
- Ha $p \leq a^k$, akkor p minden részintervallumban szitál.

$$p \leq a^k \Leftrightarrow log_a p \leq k$$

Így a rekurzív fában p legfeljebb a felső $h - \lfloor log_a p \rfloor + 1$ szinten jár be minden

csúcsot. Az alsó $\lfloor log_a p \rfloor$ szinten szintenként $\lfloor a^h/p \rfloor$ csúcsot jár be. Tehát p legfeljebb

$$
\begin{aligned}
C_p &\leq \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \sum_{k=0}^{\lfloor log_a p \rfloor} \left\lfloor \frac{a^h}{p} \right\rfloor \\
&\leq \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \sum_{k=0}^{\lfloor log_a p \rfloor} \frac{a^h}{p} \\
&= \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \frac{(\lfloor log_a p \rfloor + 1)a^h}{p} \\
&\leq \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \frac{(log_a p + 1)a^h}{p} \\
&= \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + n\left(\frac{log_a p}{p} + \frac{1}{p}\right) \\
&= \sum_{k=0}^{h-\lfloor \log_a p \rfloor} a^k + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{a^{h-\lfloor \log_a p \rfloor+1} - 1}{a-1} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&< \frac{a^{h-\lfloor \log_a p \rfloor+1}}{a-1} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&\leq \frac{2a^{h-\lfloor \log_a p \rfloor+1}}{a} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= 2a^{h-\lfloor \log_a p \rfloor} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{2a^h}{a^{\lfloor \log_a p \rfloor}} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&\leq \frac{2a^h}{a^{\log_a p - 1}} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{2a^{h+1}}{a^{\log_a p}} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= \frac{2a^{h+1}}{p} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= 2an\frac{1}{p} + \frac{n}{\ln a}\frac{lnp}{p} + n\frac{1}{p} \\
&= (2an+1)\frac{1}{p} + \frac{n}{\ln a}\frac{lnp}{p}
\end{aligned}
$$

csúcsot jár be a rekurzív fában.

Az összes prím által meglátogatott csúcsok számának becsléséhez felhasználjuk Mertens két tételét. Hivatkozás: wikipedia.

$$\sum_{p \le n} \frac{\ln p}{p} = \ln n + O(1)$$
$$\sum_{p \le n} \frac{1}{p} = \ln \ln n + O(1)$$

Így az összes prím által meglátogatott összes csúcs száma

$$\begin{aligned}
C &\le \sum_{p \le m} C_p \\
&= \sum_{p \le m} \left((2an+1)\frac{1}{p} + \frac{n}{\ln a}\frac{lnp}{p} \right) \\
&= \sum_{p \le m} (2an+1)\frac{1}{p} + \sum_{p \le m} \frac{n}{\ln a}\frac{lnp}{p} \\
&= (2an+1) \sum_{p \le m} \frac{1}{p} + \frac{n}{\ln a} \sum_{p \le m} \frac{lnp}{p} \\
&= (2an+1)\left(\ln \ln n + O(1)\right) + \frac{n}{\ln a}\left(\ln n + O(1)\right) \\
&= (2an+1)\ln \ln n + (2an+1)O(1) + \frac{n}{\ln a}\ln n + \frac{n}{\ln a}O(1) \\
&= \frac{n \ln n}{\ln a} + (2an+1)\ln \ln n + (2an+1)O(1) + \frac{n}{\ln a}O(1) \\
&= \frac{n \ln n}{\ln a} + (2an+1)\ln \ln n + O(an) + O(\frac{n}{\ln a}) \\
&= \frac{n \ln n}{\ln a} + (2an+1)\ln \ln n + O(an)
\end{aligned}$$

A RekurzívSzita program egy adott belső csúcsában egy adott prímmel többször is foglalkozik.

- Kiosztja legalább egy részintervallumra. Intervallumonként $O(1)$ időben.
- Összegyűjti a visszatérési értékbe. Egyszer $O(1)$ időben.

Valamint vannak a-tól függő inicializálási lépések, $O(a)$ időben. Egy levél csúcsban minden prímmel $O(1)$ időt foglalkozik.

Az összesítéshez számoljuk el a a részintervallumra kiosztás költségét a részintervallum csúcsában. Igy

- Egy levélben minden prímmel $O(1)$ időt foglalkozik.
- Egy belső csúcsban $O(a)$ időt tölt karbantartással, és prímenként $O(1)$ időt azzal, hogy rá osztották, vagy visszatér vele.

Tehát a teljes időigény

$$
\begin{aligned}
T &= \frac{a^h - 1}{a - 1} O(a) + CO(1) \\
&= \frac{a^h - 1}{a - 1} O(a) + O\left(\frac{n \ln n}{\ln a}\right) \\
&= O(a^h) + O\left(\frac{n \ln n}{\ln a}\right) \\
&= O(n) + O\left(\frac{n \ln n}{\ln a}\right) \\
&= O\left(\frac{n \ln n}{\ln a}\right)
\end{aligned}
$$

3.7.5. Cache-oblivious edényszita

Cache oblivious talán csak $a = 2$-re. A c program nem szép most tetszőleges a-ra. Lesz midly cache aware is, mert könnyű.

Megadunk egy konkrét implementációt a fenti RekurzívSzitára, ahol a listák egybefüggő memóriatartományok.

Az algoritmus a prímeket, amik egynél több, de nem minden részintervallumban szitálnak, minden rekurzív hívásnál újrarendezi.

3.7.6. Műveletigénye gyorsítótárral

Itt szégyenérzet nélkül fel fogom tenni, hogy egy intervallumban szitáló prímek száma az az egy-egy prím szitálásának várható értékének összege. Ami nem igaz, de senki se tud jobbat, és ad valami "elméletit", amivel össze lehet hasonlítani méréseket.

3.7.7. Feltételek a gyorsítótár méretére

3.7.8. Tetszőleges n-ig, nem csak teljes fára

3.7.9. Az igazi program fix méretű számokkal

3.7.10. Az igazi program változó méretű számokkal

Valszeg nem lesz, mert semmi hasznosat nem tudok mondani.

4. fejezet

Szimulátor

4.1. Igazi cache

Asszociativitás. Több szint. Több mag osztozik. Léteznek egészen speciális cache-ek, TLB, decoded microcode, trace cache, ...

4.2. Megoldás

4.3. Választható cache-ek

- utasítás/adat/mindkettő
- MIN/OPT.
- LRU
- FIFO, LIFO, LFU, ..., kevésbé izgalmasak
- teljes/1-2-3...-n asszociativitás
- mindenféle méretben
- egymás után akár több is

5. fejezet

Mérési eredmények

6. fejezet

Irodalomjegyzék

[1] Frigo, M., Leiserson, C. E., Prokop, H., Ramachandran, S.: Cache-oblivious algorithms, ACM Transactions on Algorithms (TALG), 8(1) (2012)., 1-22.

[2] Belady, L. A.: A study of replacement algorithms for virtual storage computers. IBM Systems Journal 5, 2 (1966), 78–101.

[3] Belady, L. A.; Nelson, R. A.; Shedler, G. S.: An anomaly in space-time characteristics of certain programs running in a paging machine. Communications of the ACM, 12(6), (1969) 349–353.

[4] Fornai, P., Iványi, A.: FIFO anomaly is unbounded. (2010) arXiv preprint arXiv:1003.1336.

[5] Mattson, R.L., Gecsei, J., Slutz, D.R., Traiger, I.L.: Evaluation Techniques for Storage Hierarchies. IBM Syst. J., 9, (1970) 78-117.

[6] Aspnes, J. (1998). Competitive analysis of distributed algorithms. In: Fiat, A., Woeginger, G.J. (eds) Online Algorithms. Lecture Notes in Computer Science, vol 1442. Springer, Berlin, Heidelberg. https://doi.org/10.1007/BFb0029567

[7] Daniel D. Sleator and Robert E. Tarjan.: Amortized efficiency of list update and paging rules. Commun. ACM 28, 2 (Feb. 1985), 202–208.

[8] Hennessy, John L.; Patterson, David A. (2011). Computer Architecture: A Quantitative Approach. ISBN 978-0-12-383872-8.

[9] Cormen, Thomas A.; Leiserson, Charles E.; Rivest, Ronald L. (2001). Algoritmusok. ISBN 963-16-3029-3.